

Context:

- The goal is to ensure thorough coverage of application testing.
- Addresses a possible human testing, not an automated testing approach.
- AI can certainly facilitate some or all of this but I have made no attempt to identify where or how. That is dependent on the app team's ability, resources and preferences.
- Even if fully manual this does not need to be a labor-intensive exercise.
- The "Example" column items are totally made up, not recommendations of any sort; just included to demonstrate intent.
- This is not intended to be exhaustive, just demonstrative.

	What	Example	Why
1	Identify and record all components/functional areas that we want to test at whatever level of granularity is reasonable/desired.	Public user: Authentication main networking meeting room breakout networking room profile page etc. Admin/other internal roles: User management Network session moderation etc.	So we have a map of the application to guide our testing and against which to track our feedback,
2	Identify and record upstream and downstream relationships between "components"	Admin networking moderation console -> main meeting room -> breakout room Can be textual, a flow diagram, etc.	So we can map any issues in one area to possible effects/issues in other related areas (e.g., an issue seen in an end user interface may be caused by functionality 2 steps downstream in an admin interface)
3	Tie user testing feedback to the components in Step 1	In today's session issues logged against either main networking room or breakout room and visible in that artifact	Allows everyone to see all issues associated with a component. Also reduces redundancy (e.g., if an issue is already entered it doesn't need to be entered again and/or can have a "tally" associated with it.

	What	Example	Why
4	Record all the above so that the info is usable, manageable and traceable	Issues in step 3 are tied to components in step 1 , and any upstream/down stream impacts can be assessed (step 2)	We want to be able to reference and maintain this information to ensure thorough testing and coverage. If it is too clunky it will fall into disuse. We can then tie these results to technical components for bug fixes, etc.

Usage:

- When user testing is happening whoever is moderating or tracking knows what sections feedback is related to
- The moderator or testers can record the results against the relevant component.
- Offline app owners can confirm all elements of the app are tested, see results per component, etc.
- Issues can be funneled back to engineering with user facing issues tied to technical components.
- AI can be included at whatever steps makes sense